

Lección 6 — Metadata Extraction and Parsing

1. Introducción General de la Lección

Metadata significa "data about data". Es decir, información adicional que describe un archivo, comunicación o recurso digital. La metadata puede revelar origen, autor, tecnologías utilizadas, geolocalización, timestamps, clasificación, software utilizado, protocolos utilizados, versiones y comportamiento de red.

Desde la perspectiva de ciberseguridad, la metadata es extremadamente valiosa para reconnaissance, threat hunting, forensic analysis, incident response, clasificación documental e identificación de tecnologías vulnerables. Los profesionales modernos de ciberseguridad deben ser capaces de extraer, interpretar y automatizar el análisis de metadata de múltiples fuentes.

2. Objetivos

Los objetivos de aprendizaje para esta lección son:

- Interpretar HTTP response headers y detectar debilidades web.
- Analizar metadata de protocolos de red.
- Interpretar headers IP/TCP y extraer información.
- Extraer metadata desde archivos PDF, Word y Excel.
- Automatizar parsing de metadata desde múltiples fuentes.
- Utilizar metadata para reconnaissance y OSINT.
- Validar configuración de seguridad basada en headers.
- Detectar información sensible en metadata documental.

3. Conceptos Técnicos Fundamentales

HTTP Response Headers

Cuando un navegador o cliente HTTP realiza una petición, el servidor devuelve contenido, código HTTP y headers. Los headers contienen información extremadamente valiosa sobre seguridad, tecnologías, políticas y configuración del servidor.

****Headers Importantes:****

- ****X-Frame-Options****: Protege contra click-jacking (valores: DENY, SAMEORIGIN)
- ****X-XSS-Protection****: Ayuda a prevenir ataques XSS
- ****X-Content-Type-Options****: Evita MIME sniffing (valor recomendado: nosniff)
- ****Referrer-Policy****: Controla información de referrer
- ****Content-Type****: Tipo de contenido
- ****Set-Cookie****: Cookies de sesión
- ****Strict-Transport-Security****: Fuerza HTTPS

- **Content-Security-Policy**: Reduce riesgo de XSS y data injection
- **Cross-Origin-Resource-Policy**: Control de CORS
- **X-Powered-By**: Puede revelar tecnologías vulnerables

Uso de requests para Obtener Headers

La librería `requests` permite obtener únicamente los headers sin descargar el contenido completo:

```
response = requests.head(url)
```

Luego se pueden leer headers específicos:

```
response.headers['Set-Cookie']  
response.headers['X-Frame-Options']  
response.headers['Referrer-Policy']
```

Interpretación de Vulnerabilidades

Desde la perspectiva ofensiva, el análisis de headers permite:

- Identificar debilidades
- Detectar malas configuraciones
- Descubrir tecnologías
- Detectar ausencia de controles

Desde la perspectiva defensiva, permite:

- Validar hardening
- Validar compliance
- Validar buenas prácticas

Protocol Headers

Los protocolos de red poseen headers estandarizados: Ethernet, IP, TCP, UDP, TLS. Estos headers contienen direcciones, flags, puertos, tamaños, control de flujo, fragmentación y TTL.

RFC (Request For Comments): Estándares de Internet que definen protocolos como TCP, IP, UDP, HTTP, TLS, IPv6.

IP Header

El header IP contiene campos como:

- **Version**: IPv4 o IPv6
- **Header Length**: Longitud del header
- **Total Length**: Longitud total del paquete
- **Identification**: Identificador único
- **Flags**: Control de fragmentación
- **Fragment Offset**: Desplazamiento de fragmento

- ****Time To Live (TTL)****: Número de hops
- ****Protocol****: Protocolo encapsulado
- ****Source Address****: IP origen
- ****Destination Address****: IP destino

****TTL**** es extremadamente útil para troubleshooting, fingerprinting, análisis de rutas y detección de anomalías.

TCP Header

TCP contiene:

- ****Source Port****: Puerto origen
- ****Destination Port****: Puerto destino
- ****Sequence Number****: Número de secuencia
- ****Acknowledgment Number****: Número de confirmación
- ****Flags****: SYN, ACK, PSH, FIN, RST
- ****Window****: Tamaño de ventana
- ****Checksum****: Validación de integridad

Los flags TCP permiten reconstruir sesiones, detectar scanning, detectar anomalías y analizar conexiones.

Pyshark y Captura de Paquetes

La librería `pyshark` funciona sobre `tshark` y permite capturar tráfico en tiempo real usando `LiveCapture()`. La captura de paquetes requiere privilegios elevados (sudo, administrador). Sin ello, la captura fallará.

Un paquete puede contener múltiples capas: ETH, IP, TCP, TLS. Python permite acceder a capas específicas, imprimir atributos y automatizar análisis.

TLS Layer

Aunque TLS cifra el contenido, todavía es posible observar versión TLS, tamaño de paquetes, timing, endpoints y metadata de sesión. Esto sigue siendo útil para forensic analysis, threat hunting, traffic analysis y detección de malware.

Extracción de Metadata desde Archivos

La metadata puede extraerse desde múltiples formatos: PDF, Word, Excel, PowerPoint. La metadata puede contener autor, creator, timestamps, clasificación, categorías, keywords, comentarios y geolocalización.

Esto permite clasificación documental, indexación, identificación de origen, investigaciones, auditoría y data discovery.

Internals de Archivos Office

Los archivos .docx, .xlsx, .pptx son realmente archivos ZIP. Dentro del ZIP existen múltiples archivos XML. El archivo `docProps/core.xml` contiene metadata crítica: creator, description, category, modified date, created date, keywords.

Metadata en PDF

Para PDF se utiliza `PyPDF2`. La metadata PDF puede revelar software creador, timestamps, autor, versión y herramientas utilizadas. También se puede verificar si el PDF está cifrado.

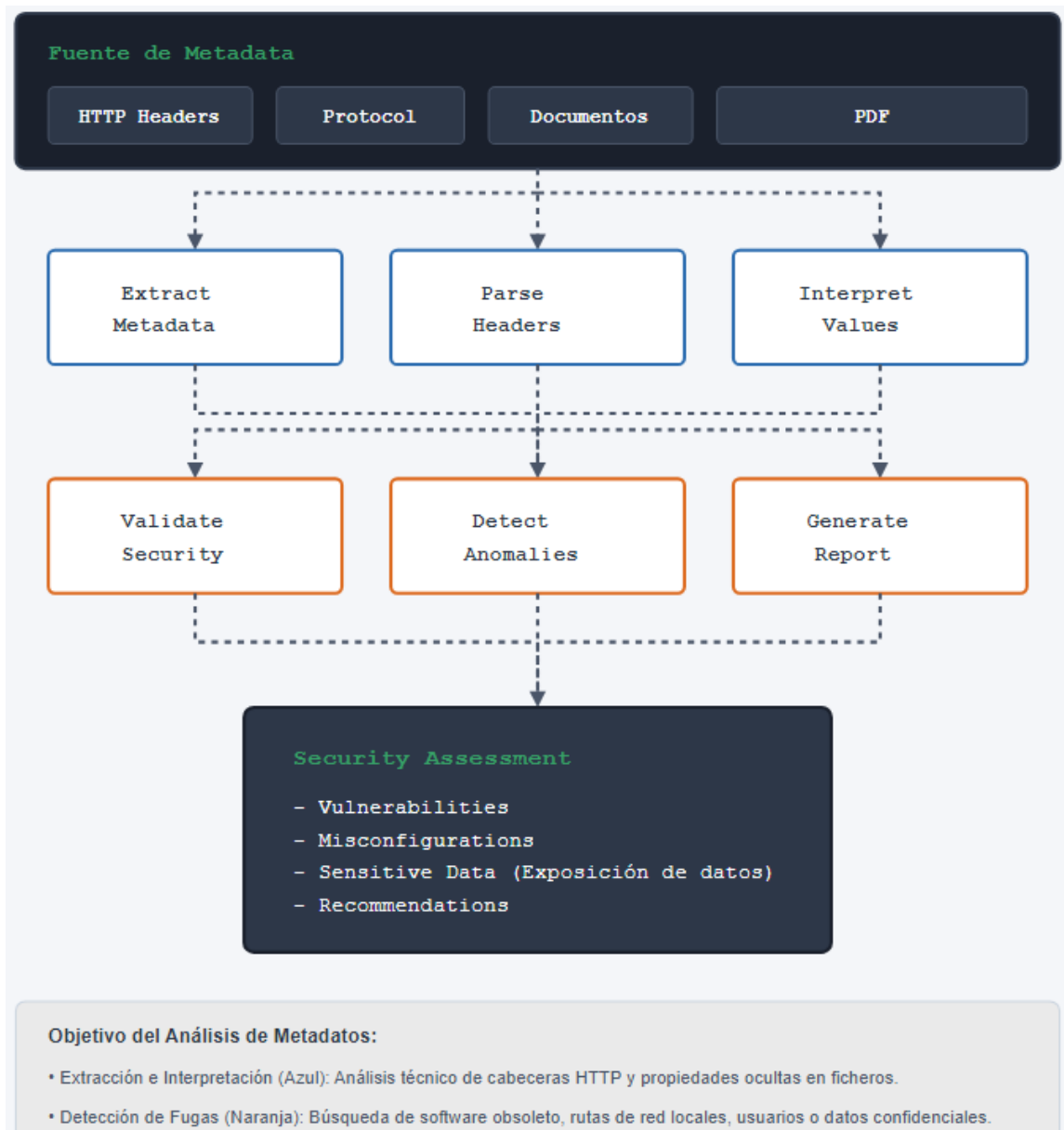
Reconnaissance usando Metadata

La metadata es extremadamente útil para reconnaissance. Permite identificar información sensible, descubrir documentos clasificados, descubrir tecnologías y detectar exposición.

Se puede automatizar detección de palabras como "Confidential" y "Classified" dentro de metadata XML. Algunos atacantes esconden payloads en metadata, usan metadata para persistencia o exfiltración. Incluso abrir un documento puede activar payloads maliciosos embebidos.

4. Arquitectura o Flujo Técnico

El flujo típico de análisis de metadata sigue esta arquitectura:



5. Herramientas Utilizadas

Herramientas de Análisis HTTP

- ****requests****: Cliente HTTP para Python

- **requests.head()**: Obtener solo headers sin contenido

Herramientas de Análisis de Protocolos

- **pyshark**: Interfaz Python a Wireshark
- **tshark**: CLI de Wireshark

Herramientas de Extracción de Metadata

- **PyPDF2**: Lectura de metadata PDF
- **zipfile**: Acceso a archivos ZIP (Office)
- **xml.dom.minidom**: Parsing de XML

Librerías Python Requeridas

```
# Librerías estándar (incluidas en Python)
# - os
# - zipfile
# - xml.dom.minidom

# Librerías externas
pip install requests
pip install pyshark
pip install PyPDF2
```

6. Scripts de la Lección

6.1 Script: http_response_header_print.py

Introducción del Script

Este script obtiene y imprime todos los HTTP response headers de una URL. Permite visualizar la configuración de seguridad del servidor.

Propósito: Obtener y mostrar HTTP headers de un servidor.

Tipo: Herramienta de análisis, reconnaissance.

Requisitos

- **Librerías Python**: `requests`
- **Instalación**: `pip install requests`
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Ninguno

Dependencias Externas

- Acceso a Internet.
- URL accesible.

Explicación Línea por Línea

```
import requests
```

Importa librería `requests`.

```
class HTTPResponseHeaderPrinter:
    def print_headers(self, url):
        response = requests.head(url)
```

Realiza HEAD request para obtener solo headers.

```
for header, value in response.headers.items():
    print(f'{header}: {value}')
```

Itera e imprime cada header y su valor.

Cómo Ejecutar el Script

```
python3 http_response_header_print.py
```

Resultado Esperado

Lista de headers HTTP del servidor:

```
Content-Type: text/html; charset=utf-8
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000
...
```

Riesgos y Consideraciones Éticas

- No hay riesgos de seguridad.
- Herramienta de diagnóstico únicamente.

Troubleshooting

- ****Timeout****: URL no accesible.
- ****ConnectionError****: Problema de red.

6.2 Script: http_response_header_interpret.py

Introducción del Script

Este script obtiene HTTP headers y los interpreta para detectar debilidades de seguridad. Valida si los headers tienen valores seguros recomendados.

****Propósito****: Analizar headers para detectar vulnerabilidades.

****Tipo****: Herramienta de análisis, vulnerability assessment.

Requisitos

- ****Librerías Python****: `requests`
- ****Instalación****: `pip install requests`
- ****Sistemas operativos compatibles****: Windows, Linux, macOS

- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Acceso a Internet.

Explicación Línea por Línea

```
import requests
```

Importa librería.

```
class HTTPResponseHeaderInterpreter:
    def interpret_headers(self, url):
        response = requests.head(url)
```

Realiza HEAD request.

```
        # Validar X-Frame-Options
        if 'X-Frame-Options' not in response.headers:
            print('⚠ X-Frame-Options ausente - vulnerable a
clickjacking')
            elif response.headers['X-Frame-Options'] not in ['DENY',
'SAMEORIGIN']:
                print('⚠ X-Frame-Options con valor débil')
            else:
                print('✓ X-Frame-Options configurado correctamente')
```

Valida cada header y reporta debilidades.

Cómo Ejecutar el Script

```
python3 http_response_header_interpret.py
```

Resultado Esperado

Reporte de configuración de seguridad:

```
✓ X-Frame-Options configurado correctamente
⚠ X-Content-Type-Options ausente
✓ Strict-Transport-Security configurado
...
```

Troubleshooting

- ****Timeout****: URL no accesible.

6.3 Script: parse_protocol_headers.py

Introducción del Script

Este script captura paquetes en tiempo real y extrae información de los headers de protocolo (IP, TCP). Permite analizar estructura de paquetes.

****Propósito**:** Capturar y analizar protocol headers.

****Tipo**:** Herramienta de análisis, packet analysis.

Requisitos

- ****Librerías Python**:** `pyshark`
- ****Instalación**:** `pip install pyshark`
- ****Sistemas operativos compatibles**:** Windows, Linux, macOS
- ****Requisitos de privilegios**:** Privilegios elevados

Dependencias Externas

- Wireshark/tshark instalado.
- Acceso a interfaz de red.

Explicación Línea por Línea

```
import pyshark
```

Importa librería.

```
class ProtocolHeaderParser:
    def do_capture(self):
        capture = pyshark.LiveCapture(interface='en0')
        capture.sniff(packet_count=5)
```

Captura 5 paquetes en tiempo real.

```
print('---- Printing all the layers of the packet')
print(capture[0])
print('\n\n---- Printing all the layer names of the
packet')
print(capture[0].layers)
```

Imprime todas las capas del primer paquete.

```
print('\n\n---- Printing only \'ip\' layer of the
packet')
print(capture[0].ip)
print('\n\n---- Printing specific attributes of the
packet')
print(capture[0].transport_layer)
print(capture[0].ip.src)
print(capture[0].ip.dst)
```

Extrae capas específicas e información.

Cómo Ejecutar el Script

```
sudo python3 parse_protocol_headers.py
```

Resultado Esperado

Detalles de paquetes capturados con información de capas.

Troubleshooting

- **Error "PermissionError"**: Ejecutar con `sudo`.

6.4 Script: `interpret_protocol_header.py`

Introducción del Script

Este script captura paquetes e interpreta los headers de protocolo. Extrae información como IP origen/destino, puertos y otros atributos.

Propósito: Analizar e interpretar protocol headers.

Tipo: Herramienta de análisis, packet interpretation.

Requisitos

- **Librerías Python**: `pyshark`
- **Instalación**: `pip install pyshark`
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Privilegios elevados

Dependencias Externas

- Wireshark/tshark instalado.

Explicación Línea por Línea

```
import pyshark
```

Importa librería.

```
class ProtocolHeaderInterpreter:
    def do_capture(self):
        capture = pyshark.LiveCapture(interface='en0')
        capture.sniff(packet_count=5)
```

Captura paquetes.

```
        print('---- Printing all the fields of the \'ip\' layer
the packet')
        print(capture[0].ip.field_names)
```

Imprime todos los campos disponibles en la capa IP.

Cómo Ejecutar el Script

```
sudo python3 interpret_protocol_header.py
```

Resultado Esperado

Información detallada de headers de protocolo.

Troubleshooting

- **Error "PermissionError"**: Ejecutar con `sudo`.

6.5 Script: metadata_extraction.py

Introducción del Script

Este script extrae metadata desde archivos PDF, Word (.docx) y Excel (.xlsx). Accede a la metadata interna de estos documentos.

Propósito: Extraer metadata desde documentos Office y PDF.

Tipo: Herramienta de análisis, metadata extraction.

Requisitos

- **Librerías Python**: `os`, `PyPDF2`, `zipfile`, `xml.dom.minidom`
- **Instalación**: `pip install PyPDF2`
- **Sistemas operativos compatibles**: Windows, Linux, macOS
- **Requisitos de privilegios**: Ninguno

Dependencias Externas

- Archivos de ejemplo: PDF, DOCX, XLSX.

Explicación Línea por Línea

```
import os
from PyPDF2 import PdfReader
import zipfile
import xml.dom.minidom
```

Importa librerías necesarias.

```
class MetadataExtractor:
    def extract_metadata(self) -> None:
        # Extract metadata for PDF file
        pdf_file = os.path.join(__location__,
            'metadata_extraction_sample.pdf')
        pdf_reader = PdfReader(pdf_file)
        print('\nMetadata for PDF file: ', pdf_reader.metadata)
        print('\nCheck if PDF file encrypted: ',
            pdf_reader.isEncrypted())
```

Extrae metadata de PDF e indica si está cifrado.

```
# Extract metadata for Word file
docx_file = os.path.join(__location__,
```

```
'metadata_extraction_sample.docx')
    word_document = zipfile.ZipFile(docx_file)
    word_extraction_path = os.path.join(__location__,
'metadata_extraction_sample_word')
    word_document.extractall(word_extraction_path)
    xml_dom =
xml.dom.minidom.parse(os.path.join(word_extraction_path,
'docProps/core.xml'))
    print('\n\nMetadata for Word file: \n',
xml_dom.toprettyxml())
```

Abre DOCX como ZIP, extrae contenido y lee XML de metadata.

```
# Extract metadata for Excel file
xlsx_file = os.path.join(__location__,
'metadata_extraction_sample.xlsx')
excel_document = zipfile.ZipFile(xlsx_file)
excel_extraction_path = os.path.join(__location__,
'metadata_extraction_sample_excel')
excel_document.extractall(excel_extraction_path)
xml_dom =
xml.dom.minidom.parse(os.path.join(excel_extraction_path,
'docProps/core.xml'))
    print('\n\nMetadata for Excel file: \n',
xml_dom.toprettyxml())
```

Mismo proceso para XLSX.

Cómo Ejecutar el Script

```
python3 metadata_extraction.py
```

Resultado Esperado

Metadata extraída de PDF, Word y Excel:

```
Metadata for PDF file: {'/Producer': 'Adobe...', '/CreationDate':
'...'}
```

```
Metadata for Word file:
<?xml version="1.0" ?>
<Properties xmlns="...">
  <dc:creator>John Doe</dc:creator>
  <dc:created>2024-01-15T10:30:00Z</dc:created>
  ...
</Properties>
```

Riesgos y Consideraciones Éticas

- La metadata puede contener información sensible.
- Verificar autorización antes de extraer metadata.

Troubleshooting

- ****Error "FileNotFoundError"****: Archivos de ejemplo no existen.

6.6 Script: `metadata_processing.py`

Introducción del Script

Este script extrae metadata de archivos Word y busca palabras clave como "Confidential" o "Classified". Permite detectar documentos sensibles.

****Propósito****: Procesar metadata y detectar información sensible.

****Tipo****: Herramienta de análisis, data discovery.

Requisitos

- ****Librerías Python****: ``os`, `zipfile``
- ****Sistemas operativos compatibles****: Windows, Linux, macOS
- ****Requisitos de privilegios****: Ninguno

Dependencias Externas

- Archivo DOCX de ejemplo.

Explicación Línea por Línea

```
import os
import zipfile
```

Importa librerías.

```
class MetadataProcessor:
    def process_metadata(self) -> None:
        docx_file = os.path.join(__location__,
        'metadata_extraction_sample.docx')
        word_document = zipfile.ZipFile(docx_file)
        word_extraction_path = os.path.join(__location__,
        'metadata_extraction_sample_word')
        word_document.extractall(word_extraction_path)
```

Abre DOCX y extrae contenido.

```
        xml_file = open(os.path.join(word_extraction_path,
        'docProps/core.xml'))
        xml_str = xml_file.read()
        print('\nMetadata for Word file:\n', xml_str)
```

Lee XML de metadata.

```
        if 'Confidential'.lower() in xml_str.lower() or
        'classified'.lower() in xml_str.lower():
```

```
print('\nThe document is tagged confidential or  
classified.')
```

Busca palabras clave en metadata.

Cómo Ejecutar el Script

```
python3 metadata_processing.py
```

Resultado Esperado

Metadata del documento y detección de clasificación:

```
Metadata for Word file:  
<?xml version="1.0" ?>  
...  
  
The document is tagged confidential or classified.
```

Troubleshooting

- ****Error "FileNotFoundError"**: Archivo no existe.**

7. Implicancias de Ciberseguridad

El análisis de metadata es fundamental para:

- ****Reconnaissance****: Obtener información del objetivo.
- ****OSINT****: Inteligencia de fuentes abiertas.
- ****Threat Hunting****: Búsqueda de amenazas.
- ****Incident Response****: Investigación de incidentes.
- ****Compliance****: Cumplimiento regulatorio.
- ****Data Loss Prevention****: Prevención de fugas.
- ****Document Security****: Seguridad documental.

8. Riesgos y Ética

Consideraciones Legales

- ****Privacidad****: Metadata puede contener información personal.
- ****Autorización****: Requiere autorización explícita.
- ****Confidencialidad****: Proteger información sensible.

Implicancias Éticas

- ****Responsabilidad****: Usar información responsablemente.
- ****Integridad****: Mantener integridad de investigación.
- ****Transparencia****: Documentar procedimientos.

9. Buenas Prácticas

1. ****Validación****: Validar configuración de seguridad.

2. ****Documentación****: Registrar todos los hallazgos.
3. ****Automatización****: Usar scripts para reproducibilidad.
4. ****Correlación****: Conectar múltiples fuentes de metadata.
5. ****Clasificación****: Clasificar información según sensibilidad.
6. ****Protección****: Proteger metadata sensible.

10. Troubleshooting

Problema	Causa Probable	Solución
"ModuleNotFoundError"	Librería no instalada	Ejecutar <code>`pip install [librería]`</code>
"FileNotFoundError"	Archivo no existe	Verificar ruta del archivo
"Timeout"	URL no accesible	Verificar conectividad
"PermissionError"	Privilegios insuficientes	Ejecutar con <code>`sudo`</code>

11. Conclusiones

Esta lección ha cubierto técnicas modernas de extracción y análisis de metadata desde múltiples fuentes. Los conceptos aprendidos son fundamentales para:

- ****Reconnaissance****: Obtener información del objetivo
- ****Vulnerability Assessment****: Identificar debilidades
- ****Incident Response****: Investigar incidentes
- ****Compliance****: Validar cumplimiento
- ****Data Discovery****: Identificar información sensible

Los scripts proporcionan herramientas prácticas para automatizar análisis de metadata. Python aparece aquí como un lenguaje extremadamente poderoso para extraer, interpretar y correlacionar información de múltiples fuentes de metadata.

La metadata, aunque frecuentemente pasada por alto, contiene información crítica que puede revelar vulnerabilidades, malas configuraciones y información sensible. Un profesional de ciberseguridad moderno debe ser capaz de extraer y analizar metadata de manera efectiva.